CIÊNCIAS DA COMPUTAÇÃO

Projeto

Teste de Software - 2026.2

Guia de instalação e execução do sistema MedAgenda (Node.js, React, MongoDB e Docker)

Prof. Bruno Rafael Araújo Vasconcelos

bruno.vasconcelos@maisunifacisa.com.br

Como rodar o MedAgenda

Este guia leva você do zero até o MedAgenda funcionando no seu computador: instalar o Docker, subir o sistema, entrar com cada perfil, chamar a API, baixar os PDFs e restaurar os dados entre uma rodada de testes e outra.

Tempo estimado: **20 a 40 minutos** na primeira vez (a maior parte é download). Depois, o sistema sobe em menos de um minuto.

1. O que você vai precisar

Item	Para quê	Obrigatório?
Docker Desktop (Windows ou macOS) ou Docker Engine com Compose (Linux)	Executar o banco, a API e a interface em contêineres	Sim
Navegador Chrome ou Firefox atualizado	Usar a interface web	Sim
Arquivo medagenda.zip ou acesso ao repositório da turma	Código do sistema	Sim
Visual Studio Code	Abrir o projeto e o terminal	Recomendado
Postman ou Insomnia	Explorar a API	Recomendado
MongoDB Compass	Ver os dados no banco	Opcional
Git	Clonar o repositório e versionar os testes do grupo	Recomendado

Computador: pelo menos **8 GB de RAM** e **10 GB livres** em disco. No Windows, a virtualização precisa estar ativada na BIOS (em geral já vem ativada).

2. Instalar o Docker

2.1 Windows 10 ou 11

1. Baixe o **Docker Desktop for Windows** em docs.docker.com/desktop.
2. Na instalação, deixe marcada a opção **Use WSL 2 instead of Hyper-V**.
3. Reinicie o computador quando o instalador pedir.
4. Abra o Docker Desktop e aguarde o ícone da baleia indicar **Engine running**.

> Se aparecer um aviso sobre o WSL, abra o PowerShell como administrador, execute `wsl --install`, reinicie e abra o Docker Desktop de novo.

2.2 macOS

1. Baixe o **Docker Desktop for Mac** escolhendo o chip correto: **Apple Silicon** (M1 a M4) ou **Intel**.
2. Arraste o Docker para Aplicativos, abra e aceite as permissões.
3. Aguarde o status **Engine running**.

2.3 Linux

Instale o Docker Engine e o plugin Compose seguindo docs.docker.com/engine/install para a sua distribuição. Para usar o Docker sem `sudo`:

```
sudo usermod -aG docker $USER
# saia da sessão e entre de novo
```

2.4 Conferir a instalação

Abra um terminal (PowerShell no Windows, Terminal no macOS e no Linux) e execute:

```
docker --version
docker compose version
```

Os dois comandos devem mostrar um número de versão. Se o segundo der erro, o Compose não está instalado (no Linux, instale o pacote `docker-compose-plugin`).

3. Obter o projeto

3.1 Pelo arquivo .zip

Extraia o **medagenda.zip** em uma pasta **sem acentos e sem espaços** no caminho, por exemplo `C:\projetos\medagenda` ou `~/projetos/medagenda`.

3.2 Pelo repositório

```
git clone <endereço do repositório informado pelo professor>
cd medagenda
```

A pasta correta é a que contém o arquivo **docker-compose.yml**:

```
medagenda/
├── docker-compose.yml
├── README.md
├── api/          (Node.js + Express + MongoDB)
│   └── src/      (routes, services, models.js, seed.js)
└── web/          (React + Vite)
    └── src/      (pages, componentes, api.js)
```

4. Subir o sistema

No terminal, **dentro da pasta medagenda**, execute:

```
docker compose up --build
```

Na primeira vez o Docker baixa as imagens do MongoDB, do Node.js e do Nginx e monta a API e a interface. Isso pode levar alguns minutos. O sistema está pronto quando aparecerem estas linhas no terminal:

```
api-1  | MongoDB conectado
api-1  | Seed: 3 médicos, 10 pacientes, 198 consultas
api-1  | API do MedAgenda em http://localhost:3000/api
```

O número de consultas pode variar um pouco, porque o histórico é gerado a partir da data de hoje.

> Deixe esse terminal aberto: ele mostra os logs do sistema enquanto você testa. Cada requisição feita à API aparece ali, com o método, a rota, o código de status e o tempo de resposta.

Se preferir liberar o terminal, suba em segundo plano com `docker compose up --build -d` e acompanhe os logs quando precisar (seção 7).

5. Conferir se está tudo funcionando

O que abrir	Endereço	O que deve aparecer
Interface web	http://localhost:8080	Tela de login do MedAgenda
Saúde da API	http://localhost:3000/api/saude	{"status":"ok","servico":"medagenda-api"}
Rota protegida sem login	http://localhost:3000/api/medicos	{"erro":"Faça login para continuar"} (é o esperado)

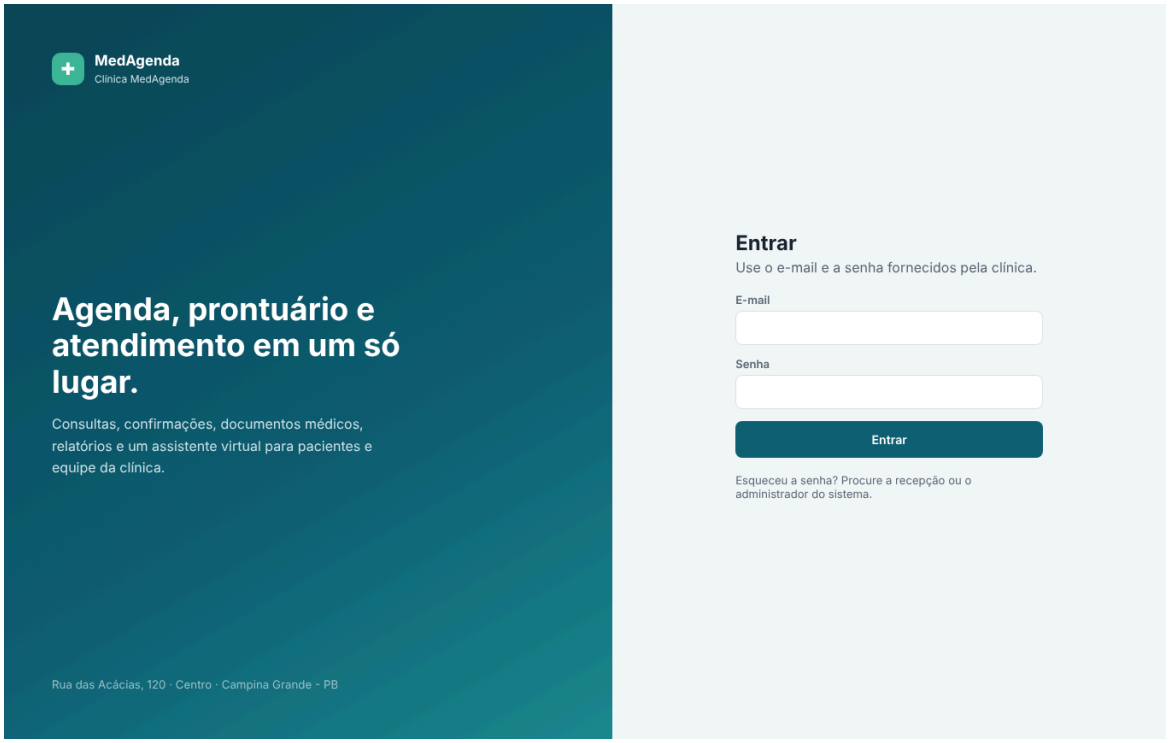


Figura 1: tela de login em http://localhost:8080

6. Contas e um passeio por perfil

Perfil	E-mail	Senha
Administrador	admin@medagenda.dev	Admin@2026
Médica (Clínica Geral)	helena.costa@medagenda.dev	Medico@2026
Médico (Cardiologia)	ricardo.alves@medagenda.dev	Medico@2026
Médica (Pediatria)	marina.duarte@medagenda.dev	Medico@2026
Secretária	joana.lima@medagenda.dev	Secretaria@2026

Perfil	E-mail	Senha
Pacientes	ana.souza@email.dev · carlos.lima@email.dev · fernanda.rocha@email.dev · luiza.costa@email.dev	Paciente@2026

Use uma **janela anônima** (ou outro navegador) para cada perfil: assim dá para ver, ao mesmo tempo, a recepção agendando e o paciente recebendo a notificação.

6.1 Administrador

Vê os indicadores da clínica, a agenda do dia, cria e gerencia contas em **Usuários** e acessa os **Relatórios** de consultas e financeiro, com exportação em PDF.

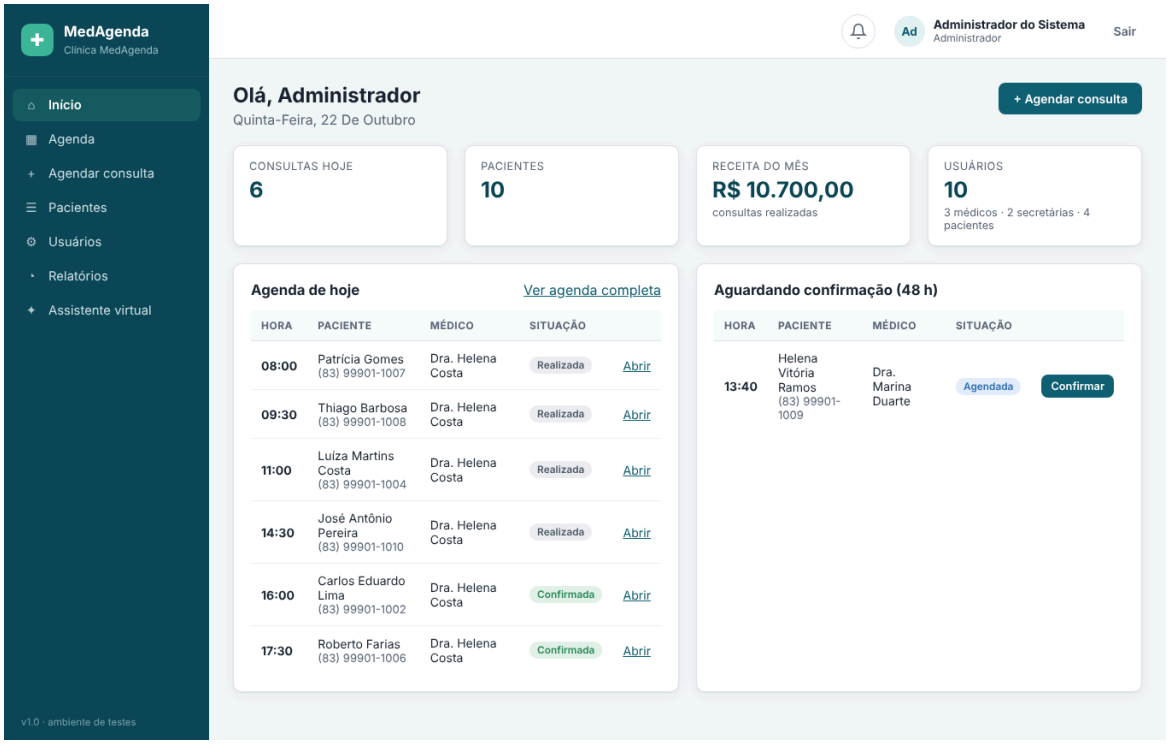


Figura 2: painel inicial do administrador

6.2 Secretária (recepção)

Cadastra pacientes, agenda consultas (horário livre ou encaixe), confirma as consultas das próximas 48 horas, registra faltas e cancela.

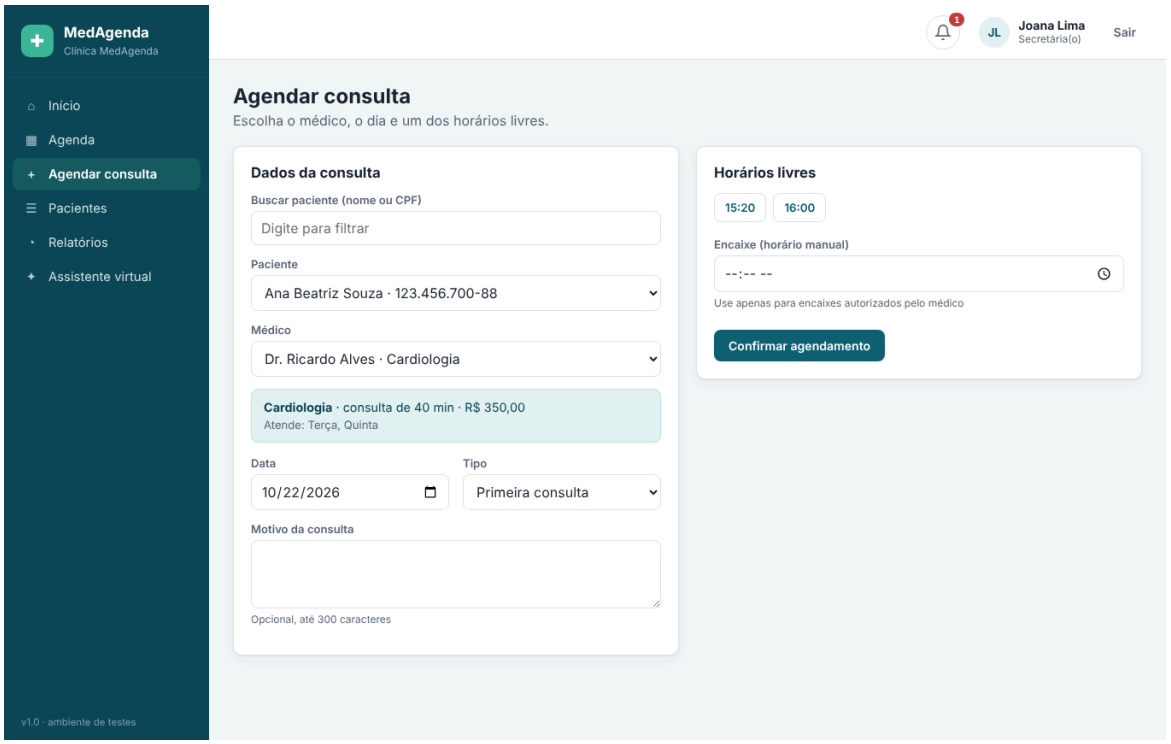


Figura 3: agendamento pela recepção, com os horários livres do médico

6.3 Médico

Vê os pacientes do dia, abre a consulta, clica em **Iniciar atendimento**, preenche o prontuário (com prescrição e dias de atestado), salva e clica em **Finalizar atendimento**. Depois disso, a receita e o atestado ficam disponíveis em PDF.

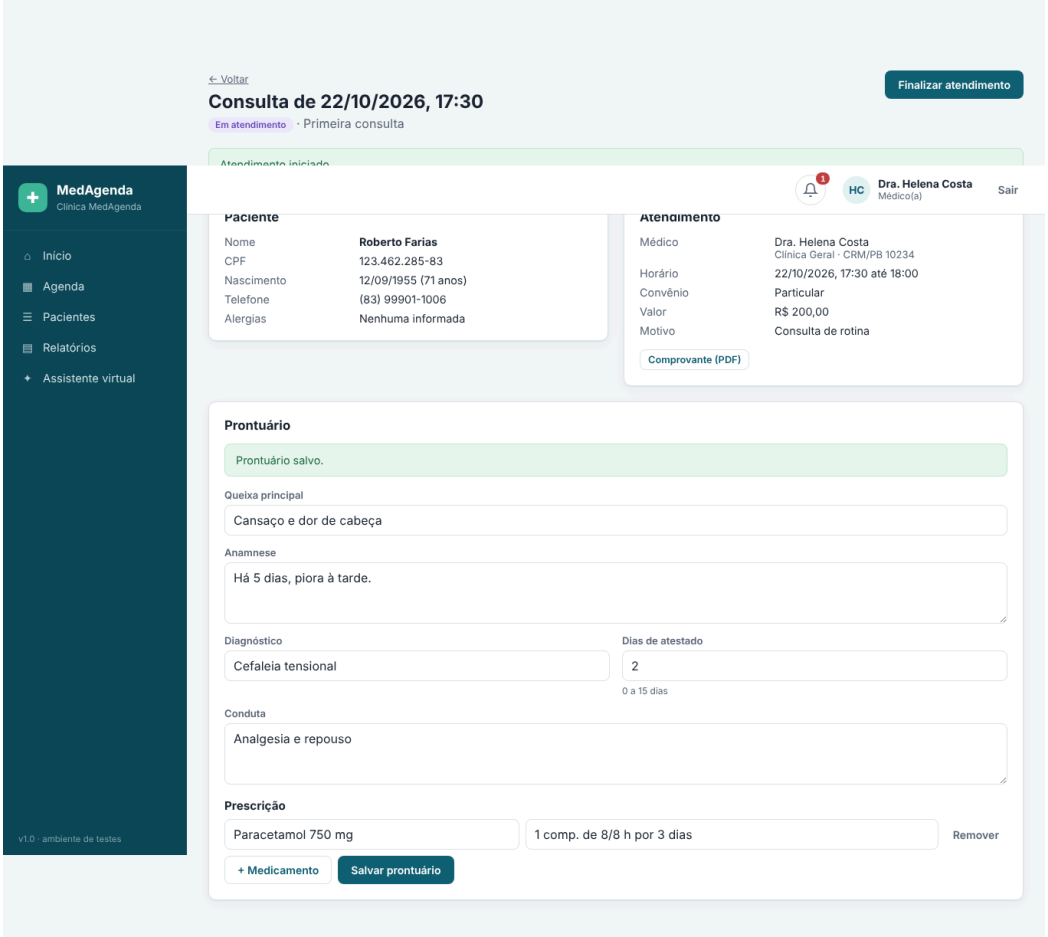


Figura 4: prontuário durante o atendimento

6.4 Paciente

Agenda e cancela as próprias consultas, baixa o comprovante, a receita e o atestado, recebe notificações (sino no topo) e conversa com o **assistente virtual**.

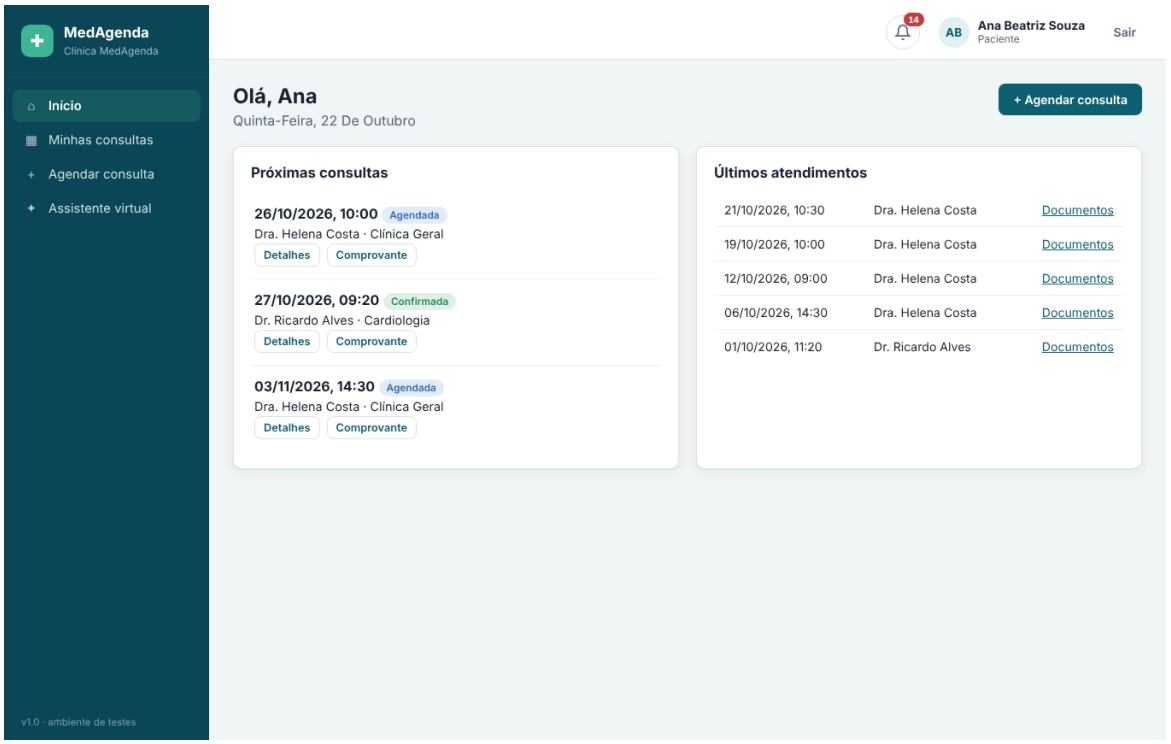


Figura 5: portal do paciente

> **Assistente virtual:** por padrão funciona sem internet, com respostas da própria aplicação. Opcionalmente, o professor pode configurar uma chave de API de IA na variável `IA_API_KEY` do docker-compose.yml; sem a chave, nada muda para os testes.

7. Comandos do dia a dia

Todos os comandos são executados na pasta **medagenda**.

Quero...	Comando
Subir o sistema (mostrando os logs)	`docker compose up --build`
Subir em segundo plano	`docker compose up --build -d`
Ver quais contêineres estão rodando	`docker compose ps`
Acompanhar os logs da API	`docker compose logs -f api`
Voltar os dados ao estado inicial (sem derrubar nada)	`docker compose exec api npm run seed`
Parar o sistema e manter os dados	`docker compose down`
Parar e apagar o banco (recria tudo na próxima subida)	`docker compose down -v`
Reiniciar só a API	`docker compose restart api`

> **Antes de cada rodada de testes**, restaure os dados com `docker compose exec api npm run seed`. Assim todos do grupo partem do mesmo estado e um teste não interfere no outro. As datas das consultas de exemplo são recalculadas a partir do dia e da hora em que o seed roda.

Com o sistema rodando em primeiro plano, **Ctrl + C** no terminal também para os contêineres.

8. Chamar a API

A lista completa de rotas está no **README.md**. Todas as rotas, exceto o login e `/api/saude`, exigem o token obtido no login, enviado no cabeçalho `Authorization: Bearer <token>`.

8.1 Com o Postman

1. Crie uma requisição **POST** para `http://localhost:3000/api/auth/login`.
2. Em **Body** → **raw** → **JSON**, envie: `{"email": "joana.lima@medagenda.dev", "senha": "Secretaria@2026"}`
3. Copie o valor de `token` da resposta.
4. Em uma nova requisição **GET** para `http://localhost:3000/api/consultas?data=AAAA-MM-DD`, vá em **Authorization** → **Bearer Token** e cole o token.
5. Dica: crie uma **coleção** com uma variável `token` por perfil (admin, secretária, médico, paciente). Isso facilita testar a matriz de permissões.

8.2 Com o PowerShell (Windows)

```
$login = Invoke-RestMethod -Method Post -Uri
http://localhost:3000/api/auth/login `
  -ContentType "application/json" `
  -Body '{"email":"joana.lima@medagenda.dev","senha":"Secretaria@2026"}'

$cab = @{ Authorization = "Bearer $($login.token)" }
Invoke-RestMethod -Uri http://localhost:3000/api/medicos -Headers $cab
```

8.3 Com o curl (macOS, Linux ou Git Bash)

```
curl -X POST http://localhost:3000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"joana.lima@medagenda.dev","senha":"Secretaria@2026"}'

TOKEN="cole_o_token_aqui"
curl http://localhost:3000/api/medicos -H "Authorization: Bearer $TOKEN"
```

8.4 Exemplo: agendar uma consulta pela API

1. Liste os médicos (`GET /api/medicos`) e os pacientes (`GET /api/pacientes`) e anote os `_id`.
2. Veja os horários livres: `GET /api/medicos/<id>/horarios?data=AAAA-MM-DD`.
3. Agende com um dos horários retornados:

```
POST /api/consultas
{
  "pacienteId": "<id do paciente>",
```

```
  "medicoId": "<id do médico>",
  "inicio": "2026-11-10T11:00:00.000Z",
  "tipo": "PRIMEIRA",
  "motivo": "Dor de cabeça frequente"
}
```

O campo `inicio` segue o padrão ISO 8601. O exemplo acima, em UTC (`Z`), corresponde a 08:00 no horário de Brasília. Também é possível informar o fuso: `2026-11-10T08:00:00-03:00`.

8.5 Baixar os PDFs

As rotas terminadas em `.pdf` devolvem o arquivo. No Postman, use **Send and Download**. No curl:

```
curl -o comprovante.pdf
http://localhost:3000/api/consultas/<id>/comprovante.pdf -H
"Authorization: Bearer $TOKEN"
curl -o consultas.pdf
"http://localhost:3000/api/relatorios/consultas.pdf?inicio=2026-10-01&fim
=2026-10-31" -H "Authorization: Bearer $TOKEN"
```

O token vale por **8 horas**. Depois disso, faça login de novo.

9. Ver os dados no banco

9.1 Com o MongoDB Compass

1. Abra o Compass e conecte em `mongodb://localhost:27017`.
2. Abra o banco **medagenda**. As coleções são **usuarios**, **pacientes**, **consultas**, **notificacaos** e **mensagemchats**.
3. Use os filtros para conferir o efeito dos testes, por exemplo `{ "status": "CANCELADA" }` em consultas.

9.2 Pelo terminal

```
docker compose exec mongo mongosh medagenda

db.consultas.countDocuments({ status: "AGENDADA" })
db.notificacaos.find().sort({ createdAt: -1 }).limit(5)
exit
```

> Use o banco para **conferir** resultados. Para **preparar** cenários de teste, prefira a interface ou a API, como um usuário real faria.

10. Rodar sem Docker (alternativa)

Use esta opção só se o Docker não funcionar no seu computador. Você vai precisar do **Node.js 20 ou superior** e do **MongoDB Community Server** rodando em `localhost:27017`.

```
# terminal 1: API (porta 3000)
cd medagenda/api
npm install
```

```
npm run seed
npm start

# terminal 2: interface (porta 5173)
cd medagenda/web
npm install
npm run dev
```

Nesse modo a interface fica em **http://localhost:5173** (e não na 8080). Configure o fuso do computador para **Brasília** para que o ambiente fique igual ao do Docker.

11. Problemas comuns

Sintoma	Causa provável	Solução
"Cannot connect to the Docker daemon" ou "error during connect"	Docker Desktop fechado ou ainda iniciando	Abra o Docker Desktop e espere o status Engine running
"port is already allocated" (3000, 8080 ou 27017)	Outro programa usa a porta (outro projeto, um MongoDB local)	Feche o outro programa ou troque a porta da esquerda no docker-compose.yml, por exemplo "8081:80", e acesse a nova porta
"WSL 2 installation is incomplete"	WSL não instalado no Windows	PowerShell como administrador: `wsl --install`, depois reinicie
"Virtualization support not detected"	Virtualização desativada na BIOS	Ative Intel VT-x ou AMD-V na BIOS do computador
"permission denied" ao rodar docker no Linux	Usuário fora do grupo docker	`sudo usermod -aG docker $USER` e entre de novo na sessão
A API repete "Aguardando o MongoDB"	O banco ainda está iniciando	Aguarde; a API tenta por cerca de 40 segundos. Se desistir, rode `docker compose up -d` de novo
A interface abre, mas o login não responde	A API não subiu	Veja `docker compose logs api` e procure a mensagem de erro
"Sessão expirada. Faça login novamente"	Token com mais de 8 horas ou banco recriado	Faça login de novo
Não aparecem horários livres	Dia em que o médico não atende, ou horários já passaram	Confira os dias de atendimento no enunciado e escolha uma data futura
"no space left on device"	Imagens e volumes antigos ocupando disco	`docker system prune` (apaga contêineres e imagens sem uso)
Comportamento estranho depois de baixar uma nova versão	Imagens antigas em cache	`docker compose build --no-cache` e depois `docker compose up`

12. Usando o MedAgenda nos testes automatizados

- Crie a pasta `tests/` na raiz do repositório do grupo, separada por tipo: `tests/api`, `tests/e2e`, `tests/bdd`, `tests/carga`, `tests/a11y`.
- Endereços: interface em `http://localhost:8080`, API em `http://localhost:3000/api`.

- Antes de executar a suíte, espere a API responder em `/api/saude` e rode o seed para partir dos dados iniciais.
- Como as consultas de exemplo dependem da data, **calcule as datas nos próprios testes** (ex.: "a próxima terça-feira") em vez de fixá-las no código.

Exemplo de passos no GitHub Actions para subir o sistema antes dos testes:

```
- uses: actions/checkout@v4
- name: Subir o MedAgenda
  run: docker compose up --build -d
- name: Aguardar a API
  run: |
    for i in $(seq 1 60); do
      curl -sf http://localhost:3000/api/saude && exit 0
      sleep 2
    done
    docker compose logs api
    exit 1
- name: Executar os testes
  run: npm test --prefix tests/api
```

Dúvidas que não estiverem neste guia: tragam para a aula ou enviem para o e-mail do professor, informando o sistema operacional e a mensagem de erro completa.